



ESTUDIO DE LA DISCOTECA “WASA WASA” PROPUESTA APP MÓVIL PARA GESTIÓN DE CLIENTES

SISTEMAS DE INFORMACIÓN
SERGIO ENRIQUE VARGAS PEDRAZA

VALENTINA GUTIERREZ PEREIRA
ANA SOFIA RODRIGUEZ NEIRA
NICOLE ARIADNA CELEMIN TRIANA
MARLON DAVID PABÓN MUÑOZ

23 DE OCTUBRE DE 2025
SEDE BOGOTÁ - FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS E INDUSTRIAL
UNIVERSIDAD NACIONAL DE COLOMBIA

CONTENIDO

1. LEVANTAMIENTO DE REQUERIMIENTOS

- 1.1 Técnicas aplicadas
- 1.2 Requerimientos Funcionales
- 1.3 Requerimientos No Funcionales
- 1.4 Reglas de Negocio 1.5 Casos de Uso
- 1.6 Matriz de Trazabilidad

2. MODELADO DEL SISTEMA

- 2.1 Diagrama de Casos de Uso
- 2.2 Diagrama de Clases
- 2.3 Diagramas de Secuencia (mínimo 2)
- 2.4 Diagrama de Actividades

3. DISEÑO DE DATOS Y ARQUITECTURA

- 3.1 Arquitectura del Sistema
- 3.2 Tecnologías Utilizadas
- 3.3 Modelo de Datos

4. PROTOTIPO FUNCIONAL (código)

- 4.1 Funcionalidades Implementadas
- 4.2 Evidencias 4.3 Casos de Prueba
- 4.4 Acceso a Base de Datos

5. DOCUMENTACIÓN TÉCNICA

- 5.1 Manual de Instalación
- 5.2 Credenciales de Acceso
- 5.3 Problemas y Limitaciones

1. LEVANTAMIENTO DE REQUERIMIENTOS

1.1 Técnicas aplicadas

Para la construcción del sistema de gestión de listas free, preventas y control de acceso mediante códigos QR, se aplicaron las siguientes técnicas de levantamiento de requerimientos:

- Entrevistas semiestructuradas

Realizadas con los principales stakeholders identificados en el Primer Entregable (administradora, organizadores, propietario, personal de acceso, contadora).

Objetivo: identificar problemas operativos, fugas de ingresos y necesidades específicas.

- Observación directa en el lugar

Se observaron procesos completos de:

- Registro de listas en WhatsApp
- Venta de preventas manual
- Validación de acceso en la puerta
- Cobro y entrega de fichas físicas de consumo

Esto permitió detectar fallos en control, tiempos, duplicidad de información y pérdida de trazabilidad.

- Análisis documental

- Procesos operativos descritos en el Primer Entregable
- Estructuras organizacionales
- Información sobre ventas y operación de eventos
- Flujos actuales y problemas reportados

- Benchmarking básico

Comparación con sistemas utilizados por otras discotecas que ya emplean QR o sistemas de registro.

Evidencias:

Pregunta	R
¿Considera usted que la propuesta de la aplicación para gestionar listas, preventas y registros es viable para Wasa Wasa?	S
¿Cómo percibe el impacto que tendría el control de aforo en tiempo real dentro de las operaciones actuales del establecimiento?	E
¿Cree que los clientes aceptarían migrar del registro por WhatsApp hacia un sistema digital más estructurado?	C
¿Qué opinión tiene sobre el uso de códigos QR para agilizar el ingreso al evento?	N
¿Considera que la aplicación podría contribuir al incremento en ventas o asistencia?	S
¿Cómo manejaría usted un posible fallo técnico del sistema durante una noche de alta afluencia?	S
¿Considera útil que la aplicación genere reportes financieros automáticos?	S
¿Cree que el personal de acceso se adaptaría fácilmente al uso del nuevo sistema?	P
¿Qué mejoras o cambios implementaría al proyecto para hacerlo más viable?	C
¿Invertiría usted en el desarrollo de esta aplicación?	S

Imagen. Preguntas entrevistas dueño Wasa Wasa

Respuesta

Sí, considero que la propuesta es viable porque actualmente muchos de los procesos se manejan de forma manual y esto genera demoras, confusiones y pérdida de información. El control de aforo en tiempo real sería muy útil, ya que permitiría tener claridad inmediata sobre la cantidad de personas dentro del establecimiento. Esto ayudaría tanto en términos de seguridad como en la experiencia del cliente. Creo que sí, siempre y cuando el sistema sea fácil de usar y no requiera muchos pasos. La mayoría de clientes ya está acostumbrada a procesos digitales, así que considero que la implementación de un sistema de control de aforo en tiempo real es viable. Me parece una idea acertada, ya que puede hacer más rápido el proceso de entrada y evitar filas innecesarias. Sin embargo, es importante asegurar que el sistema sea estable y seguro. Sí, porque una gestión más ordenada de preventas y listas puede motivar a los usuarios a planear mejor su asistencia y garantizar su ingreso con anticipación. Esto podría aumentar la satisfacción del cliente. Sería fundamental contar con un plan alternativo, como una copia local o un respaldo manual que permita continuar con la operación. No obstante, creo que con una buena infraestructura de red y seguridad, es posible. Sí, porque actualmente muchos de estos reportes se hacen manualmente y consumen tiempo. Automatizarlos permitiría tener información más precisa y disponible para la toma de decisiones. Pienso que con una capacitación sencilla y práctica podrían adaptarse sin problema. Es importante que el sistema sea claro y rápido para que su uso no interfiera con la operación. Consideraría importante incluir un módulo sencillo para casos excepcionales, como ingresos manuales o contingencias, y asegurar que la interfaz sea lo más intuitiva posible. Sí, invertiría, siempre que se presente un plan claro de desarrollo, mantenimiento y escalabilidad. Veo un potencial beneficio tanto operativo como financiero.

Imagen. Respuestas del dueño de Wasa Wasa

Transcripción de la entrevista (Dueño Wasa Wasa)

Valentina Gutierrez

Entrevistador: Valentina Gutierrez

Entrevistada: Dueño Wasa Wasa

00:01–00:16 Valentina:

¿Considera usted que la propuesta de la aplicación para gestionar listas, preventas y registros es viable para Wasa Wasa?

00:18–00:32 Dueño:

Sí, considero que la propuesta es viable porque actualmente muchos de los procesos se manejan de forma manual y esto genera demoras, confusiones y pérdida de información. Una aplicación permitiría centralizar la información y facilitar tanto el trabajo del equipo como la experiencia de los asistentes.

00:33–00:45 Valentina:

¿Cómo percibe el impacto que tendría el control de aforo en tiempo real dentro de las operaciones actuales del establecimiento?

00:46–00:59 Dueño:

El control de aforo en tiempo real sería muy útil, ya que permitiría tener claridad inmediata sobre la cantidad de personas dentro del establecimiento. Esto ayudaría tanto en temas de seguridad como en la correcta toma de decisiones durante la noche.

Valentina:

¿Cree que los clientes aceptarían migrar del registro por WhatsApp hacia un sistema digital más estructurado?

01:00–01:22 Dueño:

Creo que sí, siempre y cuando el sistema sea fácil de usar y no requiera muchos pasos. La mayoría

Imagen. Transcripción de la entrevista

1.2 Requerimientos Funcionales

ID	Nombre	Descripción	Prioridad	Fuente	¿Implementado?
RF01	Registro en lista free	Permitir que un cliente se registre mediante un link público para ingresar sin pagar cover.	Alta	Organizadores / Cliente	SI
RF02	Registro de preventa	Permitir que un cliente registre sus datos y obtenga un QR único asociado al evento.	Alta	Organizadores	SI
RF03	Generación de QR	Generar un QR único, intransferible y con un solo uso por registro.	Alta	Sistema	SI
RF04	Validación de QR	El personal de acceso debe poder escanear el QR y verificar validez en tiempo real.	Alta	Seguridad	SI
RF05	Gestión de eventos	El organizador puede crear, editar y publicar eventos en la plataforma.	Alta	Administradora / Organizadores	No
RF06	Control de aforo	El sistema debe mostrar cuántas personas han ingresado y cuántas faltan por ingresar.	Alta	Seguridad / Administración	No
RF07	Consulta de registros	El organizador puede ver cuántos se inscribieron en lista free y cuántas preventas se vendieron.	Media	Organizadores	No
RF08	Dashboard administrativo	El administrador puede ver métricas clave: ingresos, asistencia, fugas, ocupación.	Media	Administradora / Propietario	No

RF09	Invalidación automática del QR	Al leer un QR, este debe quedar inmediatamente marcado como usado.	Alta	Sistema	No
RF10	Exportación de datos	Permitir exportar reportes de asistencia e ingresos para contabilidad.	Media	Contadora	No

1.3 Requerimientos No Funcionales

ID	Categoría	Descripción	Criterio de aceptación	de ¿Cumple?
RNF01	Seguridad	Los datos personales deben ser BD cifrada y uso de HTTPS.		Parcialment e
RNF02	Rendimiento	La validación del QR debe tardar < 2 segundos.	Pruebas de carga exitosas.	No
RNF03	Disponibilidad	El sistema debe estar disponible en horarios jueves–domingo sin caídas.	Uptime $\geq 99\%$.	No
RNF04	Usabilidad	El cliente debe registrarse en minutos sin crear cuenta.	≤ 2 Flujo validado.	simple SI
RNF05	Escalabilidad	Debe ser posible usar el sistema en otras sedes sin modificar la base.	Pruebas multi-sede.	No
RNF06	Compatibilidad	Funciona en celulares Android/iOS para validación.	Pruebas móviles exitosas.	No
RNF07	Integridad	No se permite modificar registros de asistentes sin permisos.	Sistema de roles.	SI
RNF08	Trazabilidad	Cada acceso, validación y registro debe guardar timestamp.	Logs visibles en BD.	No
RNF09	Recuperación	El sistema debe recuperar operación < 5 min tras caída.	Pruebas de failover.	No
RNF10	Privacidad	Cumplimiento con normativas locales de protección de datos.	Declaración políticas.	de No

1.4 Reglas de Negocio

1. Cada QR es único y solo puede usarse una vez.
2. Un registro (lista o preventa) debe asociarse a un único documento de identidad.
3. El sistema no permite duplicados en lista free ni preventa.
4. El organizador solo puede acceder a los registros de sus eventos.
5. El administrador puede acceder a todos los datos del sistema.
6. Los registros deben cerrarse automáticamente al llegar al límite de cupos definidos.
7. La validación del QR debe marcar inmediatamente la entrada en la base de datos.
8. Los datos de ingresos solo pueden ser consultados por el rol financiero.
9. Cada registro de transacción debe conservar: usuario, fecha, hora y tipo de entrada.
10. Toda preventa debe generar QR; toda lista free NO genera QR.

1.5 Casos de Uso (mínimo 3 completos)

CU01 – Registro en Lista Free

Actor: Cliente

Precondiciones:

- El organizador generó y publicó el link.
- La lista está abierta y con cupos disponibles.

Flujo Principal:

1. Cliente abre el link del evento.
2. Visualiza datos del evento.
3. Ingresa nombre, documento y celular.
4. El sistema valida campos y duplicados.
5. Se almacena el registro.
6. Se muestra confirmación.

Flujos Alternativos:

- A1: Datos incompletos → solicitar corrección.
- A2: Registro duplicado → mostrar alerta.

Postcondiciones:

- Cliente queda registrado en lista free.

CU02 – Registro de Preventa con QR

Actor: Cliente

Precondiciones:

- El organizador configuró número de cupos y precio.
- Preventas disponibles.

Flujo Principal:

1. Cliente abre el enlace de preventa.

2. Ingresa sus datos.
3. El sistema registra venta.
4. Se genera QR único.
5. El cliente recibe el QR digital.

Alternativos:

- A1: Cupos agotados → rechazar registro.
- A2: Datos incompletos → solicitar corrección.

Postcondiciones:

- QR generado y almacenado.

CU03 – Validación de QR en el evento

Actor: Personal de acceso / Organizador

Precondiciones:

- QR válido generado previamente.

Flujo Principal:

1. El actor abre la función "Validar QR".
2. Escanea el código.
3. El sistema verifica su existencia.
4. Verifica si ya estaba usado.
5. Muestra datos del cliente (nombre, documento, tipo de entrada).
6. Marca el QR como usado.

Alternativos:

- A1: QR no existe.
- A2: QR ya usado → acceso denegado.

Postcondiciones:

- Se registra el ingreso en BD.

CU04 – Creación de Evento

Actor: Organizador

Precondiciones:

- El organizador tiene rol activo.

Flujo Principal:

1. Selecciona "Crear evento".
2. Ingresa nombre, flyer, fecha, organizador y cupos.
3. El sistema valida datos.

4. Publica el evento.
5. Genera los links correspondientes.

Postcondiciones:

- Evento visible y listo para registros.

1.6 Matriz de Trazabilidad

Requerimiento	CU01	CU02	CU03	CU04
RF01 Registro lista free	X			
RF02 Registro preventa		X		
RF03 Generación de QR		X	X	
RF04 Validación QR			X	
RF05 Gestión de eventos				X
RF06 Control de aforo			X	
RF07 Consulta registros				X
RF08 Dashboard				X
RNF01–10	Todos	Todos	Todos	Todos

2. MODELADO DEL SISTEMA

2.1 Diagrama de casos de uso

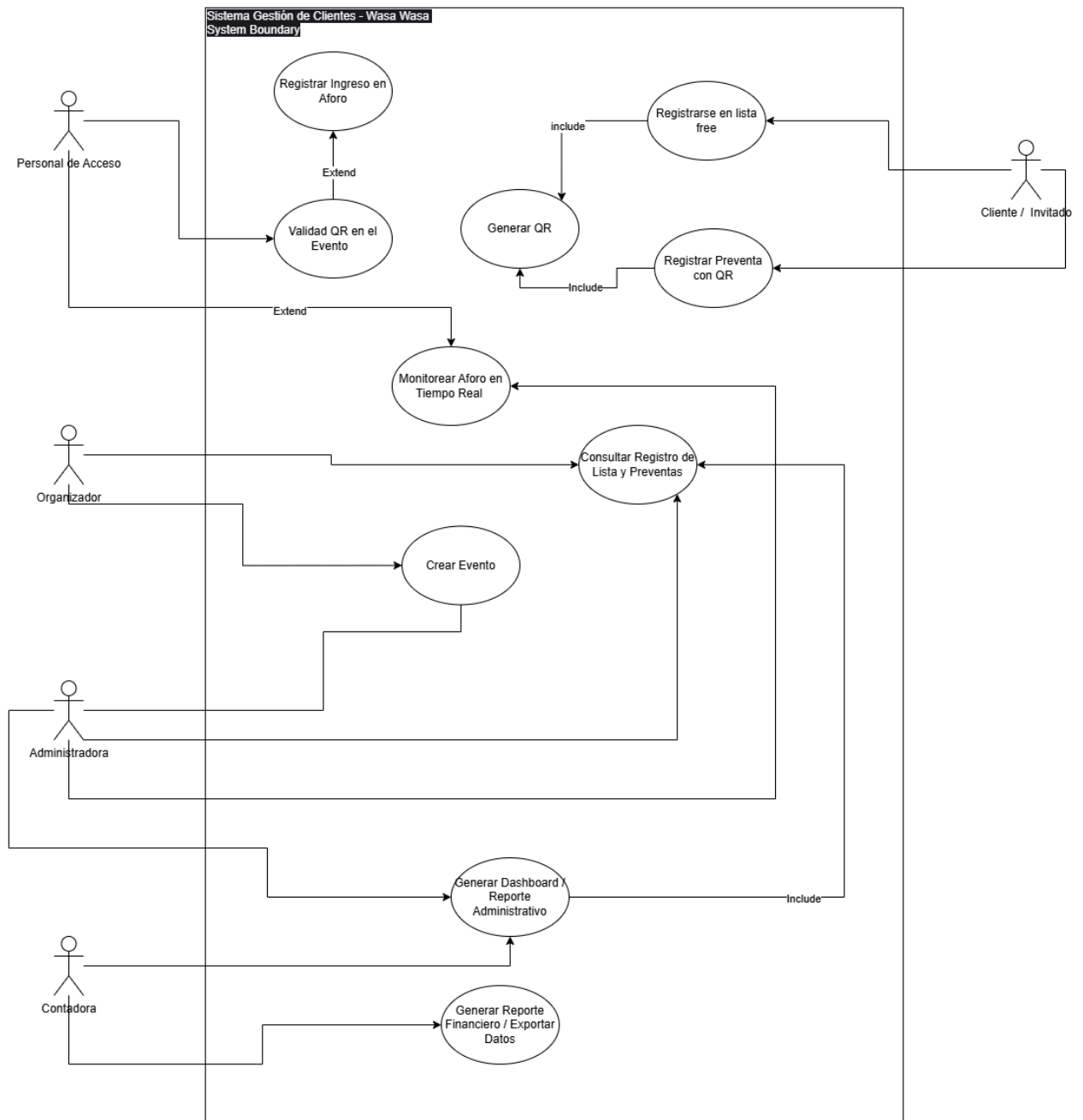


Imagen. Diagrama de casos de uso

2.1 Diagrama de Actividades

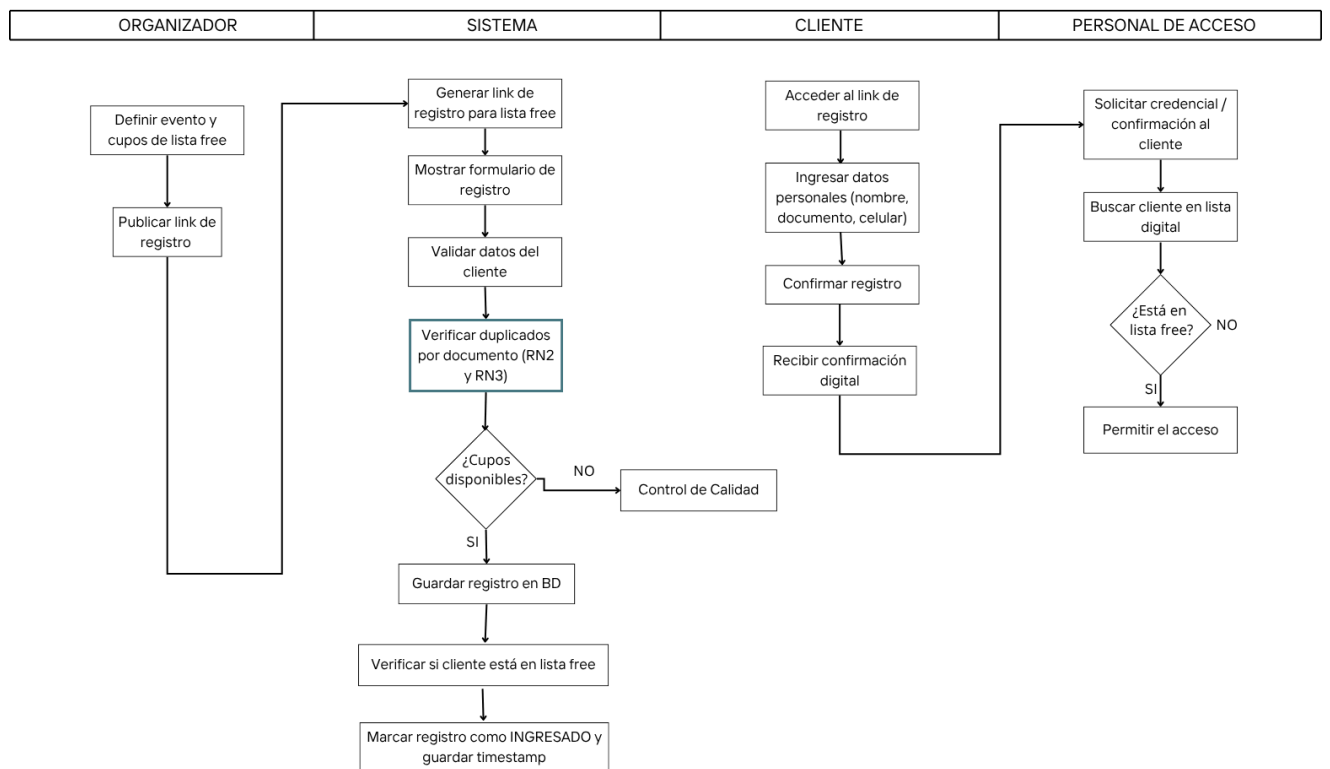


Imagen. Diagrama de actividades

3. DISEÑO DE DATOS Y ARQUITECTURA

3.1 Arquitectura del Sistema

Diagrama de Capas y Tecnologías

Tu sistema utiliza una arquitectura de tres capas bien definida, aprovechando un stack tecnológico moderno basado en JavaScript.

1. Capa de Presentación (Frontend):

- Tecnología: React.
- Responsabilidad: Construye la interfaz de usuario con la que los usuarios interactúan. Como una *Single Page Application* (SPA), se ejecuta completamente en el navegador del cliente, ofreciendo una experiencia de usuario rápida y fluida. Se comunica con la capa de lógica de negocio (el backend) a través de peticiones API (usualmente sobre HTTP/JSON) para obtener y enviar datos sin necesidad de recargar la página.

2. Capa de Lógica de Negocio (Backend):

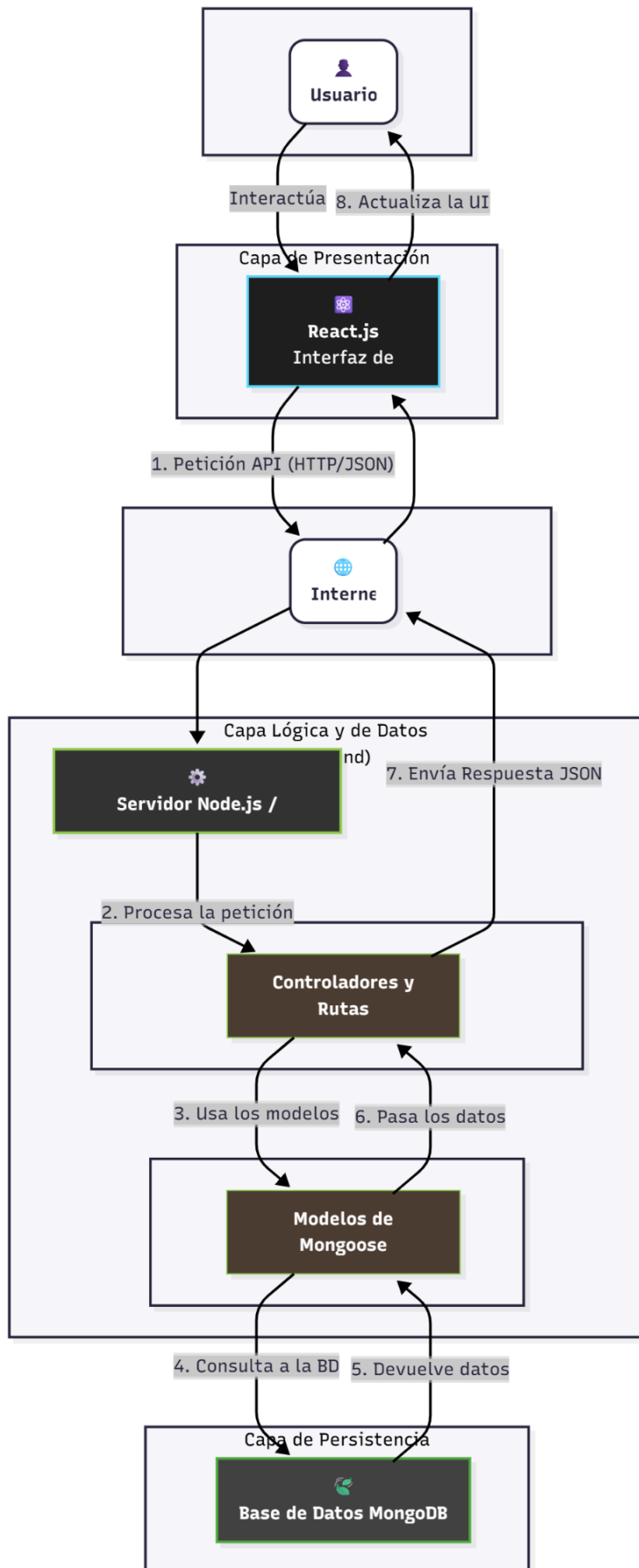
- Tecnología: Node.js.
- Responsabilidad: Sirve como el "cerebro" de la aplicación. Expone una API REST que la aplicación de React consume.

§ routes: Define los endpoints (ej: /api/events, /api/users).

§ controllers: Contiene la lógica para manejar las peticiones, validar datos, invocar a la capa de datos y enviar una respuesta al cliente.

3. Capa de Datos (Backend):

- Tecnología: Node.js con una librería como Mongoose para interactuar con MongoDB.
- Responsabilidad: Gestiona la persistencia de los datos. Define los esquemas y modelos de datos (ej: el modelo de un "Evento" o un "Usuario") y se encarga de todas las operaciones contra la base de datos (CRUD: Crear, Leer, Actualizar, Eliminar). Abstrae la complejidad de las consultas a la base de datos.



Patrón Arquitectónico Usado

El patrón principal es Model-View-Controller (MVC), adaptado a una arquitectura de cliente-servidor desacoplada:

- **Model (Modelo):** Los esquemas definidos en `server/models` con Mongoose, que representan los datos en la base de datos MongoDB.
- **View (Vista):** Los componentes de React en el directorio `client`, que renderizan la interfaz de usuario.
- **Controller (Controlador):** La lógica en los archivos de `server/controllers` y `server/routes` en Node.js/Express, que manejan las peticiones del cliente (la Vista) y las conectan con el Modelo.

Justificación

La elección de esta arquitectura y del stack tecnológico (conocido como MERN: MongoDB, Express.js, React, Node.js) es una de las más populares y robustas para aplicaciones web modernas, y se justifica por las siguientes razones:

- **Desacoplamiento Total entre Cliente y Servidor:** Al tener el frontend (React) separado del backend (Node.js), ambos pueden desarrollarse, probarse y desplegarse de forma independiente. El único contrato entre ellos es la API. Esto permite que el frontend pueda ser reemplazado por otra tecnología (una app móvil, por ejemplo) sin afectar al backend.
- **Ecosistema Unificado de JavaScript:** Usar JavaScript/TypeScript en todo el stack (React en el frontend, Node.js en el backend e incluso para las consultas en MongoDB) simplifica el desarrollo. Los desarrolladores no necesitan cambiar de contexto entre diferentes lenguajes, lo que aumenta la productividad y facilita el compartir código o lógica si es necesario.
- **Flexibilidad de Datos con MongoDB:** MongoDB es una base de datos NoSQL orientada a documentos, que almacena los datos en un formato similar a JSON (llamado BSON). Esto se integra perfectamente con JavaScript, ya que los datos no necesitan ser transformados entre formatos tabulares (como en SQL) y objetos de JavaScript. Permite una gran flexibilidad para evolucionar el esquema de datos a medida que la aplicación crece.
- **Alto Rendimiento y Escalabilidad:** Node.js es conocido por su modelo de E/S (Entrada/Salida) no bloqueante, lo que lo hace ideal para construir APIs rápidas y eficientes que deben manejar muchas conexiones simultáneas. Combinado con React, que optimiza las actualizaciones del DOM, se logra una aplicación de alto rendimiento.

3.2 Tecnologías Utilizadas

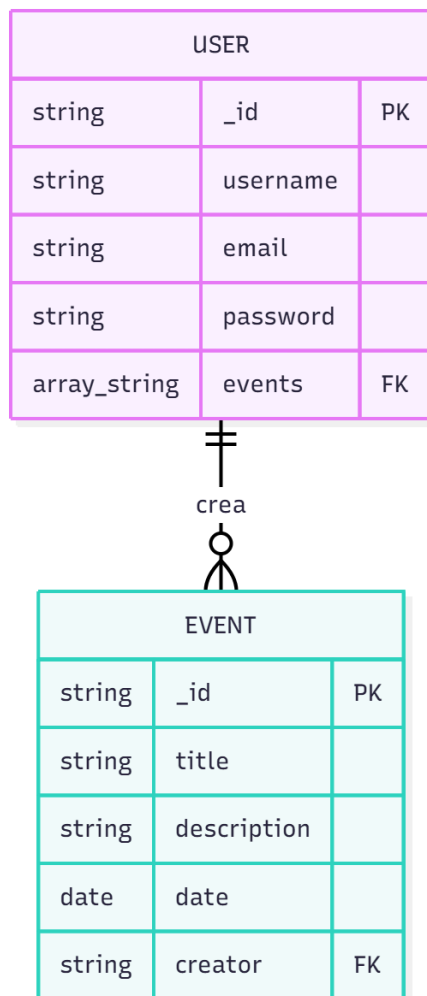
A continuación, se detallan las tecnologías clave que conforman la arquitectura de la aplicación de gestión de eventos.

Tabla de Tecnologías

Componente	Tecnología	Descripción y Justificación de Uso
Frontend	React.js	Es una biblioteca de JavaScript para construir interfaces de usuario interactivas y rápidas. Se eligió por su arquitectura basada en componentes, que facilita la reutilización de código y la mantenibilidad. Su ecosistema y popularidad aseguran un amplio soporte y la capacidad de crear una <i>Single Page Application</i> (SPA) moderna y eficiente.
Backend	Node.js y Express.js	Node.js es un entorno de ejecución de JavaScript del lado del servidor que permite construir APIs rápidas y escalables gracias a su modelo de E/S no bloqueante. Express.js es un framework minimalista sobre Node.js que simplifica la creación de rutas y la gestión de peticiones HTTP. Juntos, son ideales para construir el backend de una aplicación MERN.
Base de Datos	MongoDB (con Mongoose)	Es una base de datos NoSQL orientada a documentos. Se seleccionó porque su formato de almacenamiento (BSON, similar a JSON) se integra perfectamente con JavaScript, eliminando la necesidad de traducir datos entre el backend y la base de datos. Mongoose se utiliza como ODM (Object Data Modeling) para facilitar la interacción y definir esquemas de datos claros.
Herramienta BI	No implementada	Actualmente, el proyecto no integra una herramienta de Business Intelligence (BI) dedicada. Sin embargo, la arquitectura está preparada para ello. Los datos almacenados en MongoDB podrían ser conectados a herramientas como MongoDB Charts, Tableau o Microsoft Power BI para generar análisis visuales sobre la popularidad de los eventos, tendencias de registro de usuarios y métricas de participación.

3.3 Modelo de Datos

- Diagrama Entidad-Relación



- Diccionario de datos

Entidad: User

Campo (Atributo)	Tipo de Dato	Descripción
_id	ObjectId	Identificador único autogenerado por MongoDB.
username	String	Nombre de usuario para el inicio de sesión.
email	String	Correo electrónico único del usuario.
password	String	Contraseña encriptada para la seguridad de la cuenta.

events	Array<ObjectId>	Lista de IDs que referencia a todos los eventos creados por este usuario.
--------	-----------------	---

Entidad: Event

Campo (Atributo)	Tipo de Dato	Descripción
_id	ObjectId	Identificador único del evento.
title	String	Título del evento.
description	String	Descripción detallada del evento.
date	Date	Fecha programada para el evento.
creator	ObjectId	ID que referencia al usuario que creó el evento.

- Script SQL de creación

[User.js](#)

```
import bcrypt from 'bcryptjs';

import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['organizador', 'trabajador'], required: true },
});

// Encriptar contraseña antes de guardar

userSchema.pre('save', async function (next) {
```

```

    if (!this.isModified('password')) return next();

    this.password = await bcrypt.hash(this.password, 10);

    next();
  });

  userSchema.methods.comparePassword = function (candidatePassword) {
    return bcrypt.compare(candidatePassword, this.password);
  };

  const User = mongoose.model('User', userSchema);

  export default User;

```

Event.js

```

const mongoose = require('mongoose');

const eventSchema = mongoose.Schema(
  {
    title: { type: String, required: [true, 'Por favor añade un título'] },

    description: { type: String, required: [true, 'Por favor añade una descripción'] },

    date: { type: Date, required: [true, 'Por favor añade una fecha'] }

  },
  { timestamps: true }
);

```

```
module.exports = mongoose.model('Event', eventSchema);
```

4. PROTOTIPO FUNCIONAL (CÓDIGO)

4.1 Funcionalidades implementadas

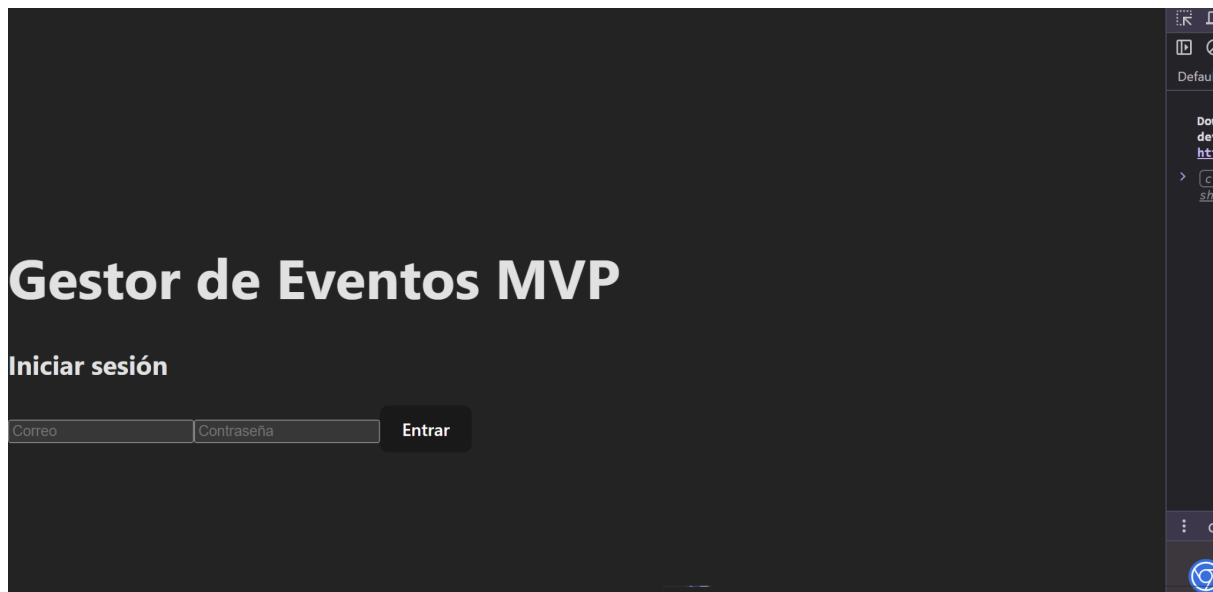
4.2 Evidencias

4.3 Casos de prueba

4.4 Acceso a base de datos

5. DOCUMENTACIÓN TÉCNICA

Pantalla de Login



Pantalla de eventos

Gestor de Eventos MVP

Próximos Eventos

Fiesta Universitaria

31/10/2025

Tardeo para estudiantes...

[Ver Detalles y Registrarse](#)

Concierto Indie

20/11/2025

Bandas locales...

[Ver Detalles y Registrarse](#)

pantalla registro en lista free o preventa

Gestor de Eventos MVP

Fiesta Universitaria

Fecha: 31/10/2025, 13:00:00

Tardeo para estudiantes

Regístrate en la Lista Free

Nombre:

Email:

[Registrarme](#)

pantalla qr generado

Gestor de Eventos MVP

¡Registro Exitoso!

Gracias por registrarte. Muestra este código QR en la entrada del evento.



[Descargar mi QR](#)

[Volver al inicio](#)

Estructura de base de datos

 Search Namespaces

▼ **eventos**

events

| **registrations**

users

▶ sample_mflix